

PISA: Efficient Precision-Slice Framework for LLMs with Adaptive Numerical Type

Ning Yang^{1,2}, Zongwu Wang^{1,2}, Qingxiao Sun³, Liqiang Lu⁴ and Fangxin Liu^{1,2,*}

¹Shanghai Jiao Tong University, ²Shanghai Qi Zhi Institute, ³China University of Petroleum-Beijing, ⁴Zhejiang University

*Corresponding Author {yn937391832, liufangxin}@sjtu.edu.cn

Abstract—Large language models (LLMs) have transformed numerous AI applications, with on-device deployment becoming increasingly important for reducing cloud computing costs and protecting user privacy. However, the astronomical model size and limited hardware resources pose significant deployment challenges. Model quantization is a promising approach to mitigate this gap, but the presence of outliers in LLMs reduces its effectiveness. Previous efforts addressed this issue by employing compression-based encoding for mixed-precision quantization. These approaches struggle to balance model accuracy with hardware efficiency due to their value-wise outlier granularity and complex encoding/decoding hardware logic. To address this, we propose PISA (Precision-Slice Framework), an acceleration framework that exploits massive sparsity in the higher-order part of LLMs by splitting 16-bit values into a 4-bit/12-bit format. Crucially, PISA introduces an early bird mechanism that leverages the high-order 4-bit computation to predict the importance of the full calculation result. This mechanism enables efficient computational skips by continuing execution only for important computations and using preset values for less significant ones. This scheme can be efficiently integrated with existing hardware accelerators like systolic arrays without complex encoding/decoding. As a result, PISA outperforms state-of-the-art precision-aware accelerators, achieving a $1.3\text{--}4.3\times$ performance boost and $14.3\text{--}66.7\%$ greater energy efficiency, with minimal model accuracy loss. This approach enables more efficient on-device LLM deployment, effectively balancing computational efficiency and model accuracy.

I. INTRODUCTION

Transformer-based large language models (LLMs) have achieved remarkable success [1], but their large model sizes and high computational demands pose significant challenges. For instance, GPT-175B requires at least 350GB of storage for FP16 inference, equivalent to five 80GB A100 GPUs [2].

Quantization [3]–[7] is one of the most effective ways to reduce LLM inference costs. By quantizing weights and activations to lower bit-widths, both memory and bandwidth demands can be greatly reduced, while accelerating computations. Currently, LLM weights can be quantized to 4-bit with minimal accuracy impact [4], [6], [8]. However, quantizing activations remains challenging due to outlier issues. A small fraction of activation values with extremely large magnitudes significantly impacts model accuracy, making it difficult to

apply low bit-width (4-bit) activation quantization without sacrificing model accuracy.

Previous works, such as OLAcel [9], GOBO [10], and OliVe [11], have proposed precision-aware quantization with dedicated hardware support. OLAcel and GOBO use mixed-precision quantization (4-bit for normal values, 16-bit for outliers) to achieve near 4-bit average precision with negligible accuracy loss, but they treat outliers as sparse matrices, requiring additional codebooks and complex control logic. OliVe introduces outlier-victim pair encoding with localized outlier storage to ensure aligned memory access, but it limits outlier bit-width extension and compromises hardware efficiency by adding encoders and decoders. While these methods effectively separate normal values from outliers, they miss opportunities for further energy reduction and accuracy improvement by ignoring the inherent sparsity in the higher-order bits of quantized values [12]–[14]. For example, an INT8 value can be split into two 4-bit parts, where the high-order part shows greater sparsity than the original value.

Motivated by this observation, we propose PISA (Precision-Slice Framework with Adaptive Numerical Type), which splits model data into 4-bit high-order and 12-bit low-order parts for mixed-precision representation using an interleaved computing architecture. PISA focuses on high-order 4-bit computation, exploiting the inherent sparsity in quantization parameters through an early bird mechanism. This mechanism predicts the importance of full calculations using partial sums from high-order computation: important results are completed, while less significant ones are skipped and replaced with a preset value. By eliminating unnecessary calculations, PISA significantly reduces computational energy and time. Rather than relying on complex encoder/decoder configurations, PISA ensures efficient storage alignment through interleaved data management and maintains model accuracy with optimized thresholds and presets.

We make the following contributions in this paper:

- We propose PISA, a precision-slice framework based on data distribution and bit-level sparsity. Using a 4bit-12bit unbalanced splitting, PISA exploits inherent high-bit sparsity to reduce storage requirements and enhance computational efficiency while preserving accuracy.
- We introduce an early bird mechanism that leverages the partial sum from the high-order 4-bit computation

This work was supported by the National Key Research and Development Program of China (2024YFE0204300), National Natural Science Foundation of China (Grant No.62402311), Natural Science Foundation of Shanghai (Grant No.24ZR1433700).

to predict the importance of the full calculation. This mechanism enables efficient computational skips for less significant values, further reducing computational costs without compromising model accuracy.

- We design a hardware-friendly, memory-interleaved computing architecture that organizes PE arrays through precision-aware processing. This architecture efficiently handles the 4bit-12bit split representation and implements the early bird mechanism using a simple comparator, significantly accelerating computation.
- We implement and evaluate PISA, demonstrating its superiority over existing quantization-based encoding schemes and hardware accelerators. Evaluations show significant improvements in both efficiency and overall performance, highlighting PISA's effectiveness and scalability in real-world LLM applications.

II. BACKGROUND

A. Outliers within Transformer Models

Transformer model parameters typically follow a Gaussian-like distribution, suitable for uniform quantization [11], [13]. However, a small fraction of parameters, termed outliers, exhibit extremely large magnitudes. Figure 1(a) shows the distribution of activation and weight parameters for a single GPT2-XL layer. While 99.67% of parameters fall within (3σ) of a fitted Gaussian distribution, outliers extend to tens or even hundreds of standard deviations.

These outliers, despite their rarity, significantly impact overall model accuracy. We experimentally remove the largest part of the parameter values, as shown in Figure 1(b). The accuracy result demonstrates that removing as few as 0.001% of outliers can substantially degrade inference accuracy.

Key Insight 1: Outliers are crucial for preserving model accuracy, making their proper management and accurate representation essential for effective model compression.

To maintain accuracy, outliers often require higher-bit representation (8 or 16 bits) [15]. However, this approach introduces bit-level redundancy in data representation. While most data falls within a range that could be represented with lower bitwidths, the presence of outliers forces the use of a uniform higher precision for all parameters, resulting in significant bit redundancy. Figure 2 illustrates that activations and weights stored in INT16 format, most parameters exhibit massive bit redundancy in their high-order part, particularly in the top 4 bits. This uniform high precision for all parameters leads to unnecessary storage and computational overhead.

Key Insight 2: Redundancy from outliers can be eliminated or mitigated by separately storing and processing the high-order and low-order parts of data representation.

The challenge lies in developing a method to effectively identify and handle outliers without compromising model accuracy. Traditional approaches struggle to balance preserving critical outlier information with efficient compression. In summary, outliers in transformer models present both challenges and opportunities for compression. This calls for the

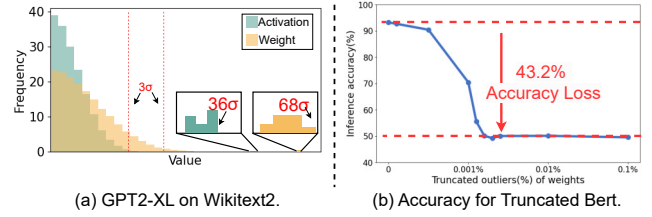


Fig. 1: (a): Outlier Comparison GPT2-XL on Wikitext-103. (b): Model Accuracy of Truncated Bert on SST-2.

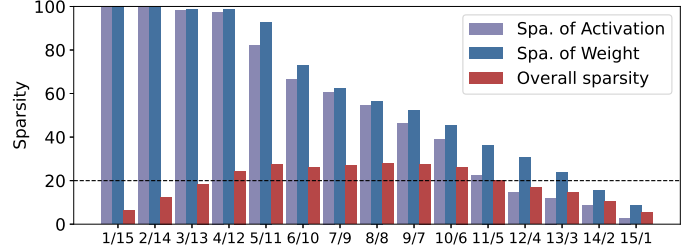


Fig. 2: Activation, weight, and overall sparsity across precision-slice methods. ‘x/y’ denotes slicing a 16-bit parameter into x-bit and y-bit parts and computing their sparsity.

efficient design that adaptively handle parameters based on their importance to overall model performance.

B. Transformer Accelerators

Previous works [4]–[6], [16]–[19] have proposed various quantization/architecture co-design methods to address the activation outlier challenge in LLMs. Several quantization algorithms smooth out the activation distribution by scaling down outlier magnitudes, enabling uniform quantization. However, encoding outliers with the same low bit-width as normal values fails to provide sufficient accuracy.

To maintain model accuracy, several outlier-aware architectures have been proposed. For instance, OLAccel [9] and GOBO [10] store high-precision outliers separately from normal values and employ dedicated processing units for outliers. However, these approaches face challenges due to variable-length outlier storage resulting from random occurrence, leading to unaligned memory access. Additionally, they require complex control logic to index outliers and orchestrate computation between normal and outlier values. OliVe [11] attempts to address these issues by locally storing outlier values and allocating co-located bit-width for indexing. While this approach achieves aligned memory access and reduces control overhead, it can only extend limited bit-width to represent outlier values, potentially leading to insufficient precision and accuracy loss. Moreover, to accommodate outlier values with local storage and random occurrence, OliVe extends all computing units to enable outlier processing, which adversely impacts its hardware efficiency.

Despite reducing computational overhead, these compression techniques increase hardware complexity. Complex encoding and decoding processes introduce additional computational costs, diminishing the potential benefits of mixed-precision quantization. Figure 3 shows that encoding/decoding

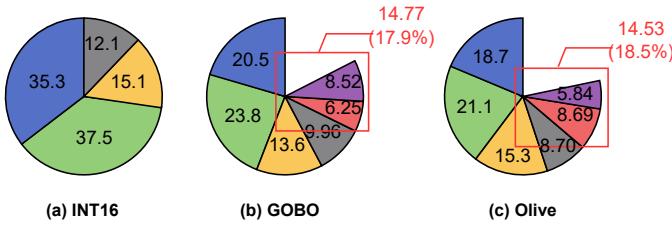


Fig. 3: Latency breakdown for different quantization methods.

consumes nearly 20% of latency in quantization-based accelerators, significantly offsetting acceleration gains.

Key Insight 3: Current mixed precision acceleration methods, despite their potential, often compromise overall performance due to the trade-off between precision and hardware complexity. This highlights the need for more efficient approaches that better balance accuracy and acceleration.

III. PISA ALGORITHM

A. Main Idea and Design Overview

Transformers contain non-ignorable outliers that require high bit-widths to maintain accuracy, leading to significant computational and storage redundancy. Existing compression-based approaches attempt to reduce this redundancy by isolating or handling outliers. However, these methods introduce serialization and deserialization overheads that offset compression benefits and increase computational or storage burdens.

To this end, we propose an outlier-aware acceleration scheme that avoids complex overhead. Our approach quantizes weights and activations into 16-bit integers, ensuring comprehensive support for a wide range of model sizes and architectures. As illustrated in Figure 2, slicing the quantized values reveals considerable overall sparsity, even for unfavorably quantized activation value weights. After thorough analysis, we opted for a 4-12 bit partitioning paradigm: 4 bits for the higher-order part and 12 bits for the lower-order part.

Such a partitioning offers several advantages. The 4-bit higher-order part exhibits over 95% sparsity, even for activations, opening new avenues for efficient quantization and compression. It also helps identify important outliers, enabling selective computation skipping for faster processing. The 12-bit lower-order part can be divided into three 4-bit segments, naturally aligning with memory and hardware, eliminating the need for complex alignment schemes [20].

Based on these insights, we introduce PISA (Outlier-aware Precision-Slice), a framework that treats quantized values as two independent parts—high-precision and low-precision—processed in parallel through precision-slicing. PISA splits 16-bit quantized model parameters into 4-bit and 12-bit segments, both participating in parallel computation in 4-bit format. Through interleaved computation and simple comparators, PISA leverages the sparsity of higher-order parts to achieve energy savings and performance acceleration.

B. Precision-Slice Computation of PISA

Based on the 4/12 bit division, we propose a precision-slicing computation flow with an **Early Bird** mechanism. This

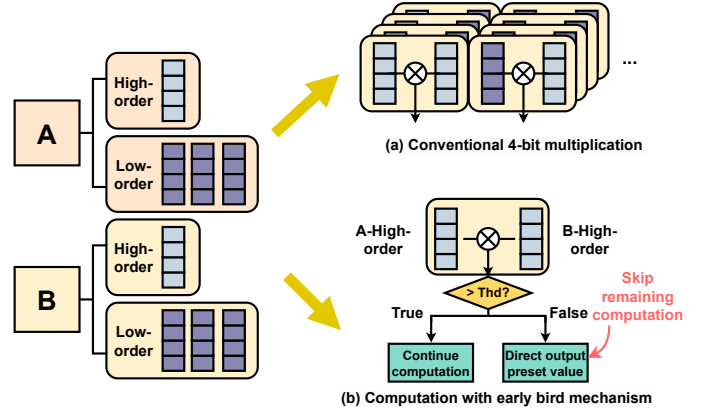


Fig. 4: PISA Computation Mechanism: (a) Conventional 4-bit multiplication, (b) PISA’s early bird mechanism.

approach leverages the properties of outliers and the sparsity of the higher-order part to accelerate LLM computation, as illustrated in Figure 4.

For 16-bit numbers A and B, we split and execute multiplication in 4-bit format (Figure4(a)). The early bird mechanism (Figure4(b)) leverages the partial sum from the high-order 4-bit computation to predict the importance of the full calculation. When computing the product of A and B, we first calculate the product of their higher-order parts and perform a threshold detection. If this partial result exceeds a predetermined threshold, we consider the calculation “significant” to the neural network and proceed to complete the full computation. Conversely, if the result is below or equal to the threshold, we deem it less important, mask the result to a preset value, and skip the remaining calculations.

This approach enables efficient computational skips for less significant values, substantially reducing overall computational costs without compromising model accuracy. To optimize the trade-off between efficiency and accuracy, we determine the optimal threshold and preset values through offline layer-wise validation. This process involves running the validation set and fine-tuning these parameters to minimize any potential accuracy loss while maximizing computational savings.

By combining the early bird mechanism with the intrinsic sparsity of higher-order bits, PISA accelerates LLM computation, improving the performance of both attention and linear modules in the Transformer.

IV. PISA ARCHITECTURE

This section describes integrating PISA into a systolic array architecture with fixed weights. With minimal architectural modifications, PISA delivers substantial acceleration benefits. Notably, it requires no additional encoders/decoders or extra storage overhead, offering a distinct advantage over other accelerators.

A. Overview

As illustrated in Figure 5(a), PISA architecture consists of multiple PE pages, each containing multiple PE arrays capable of performing 16-bit computations in a 4-bit format. Each

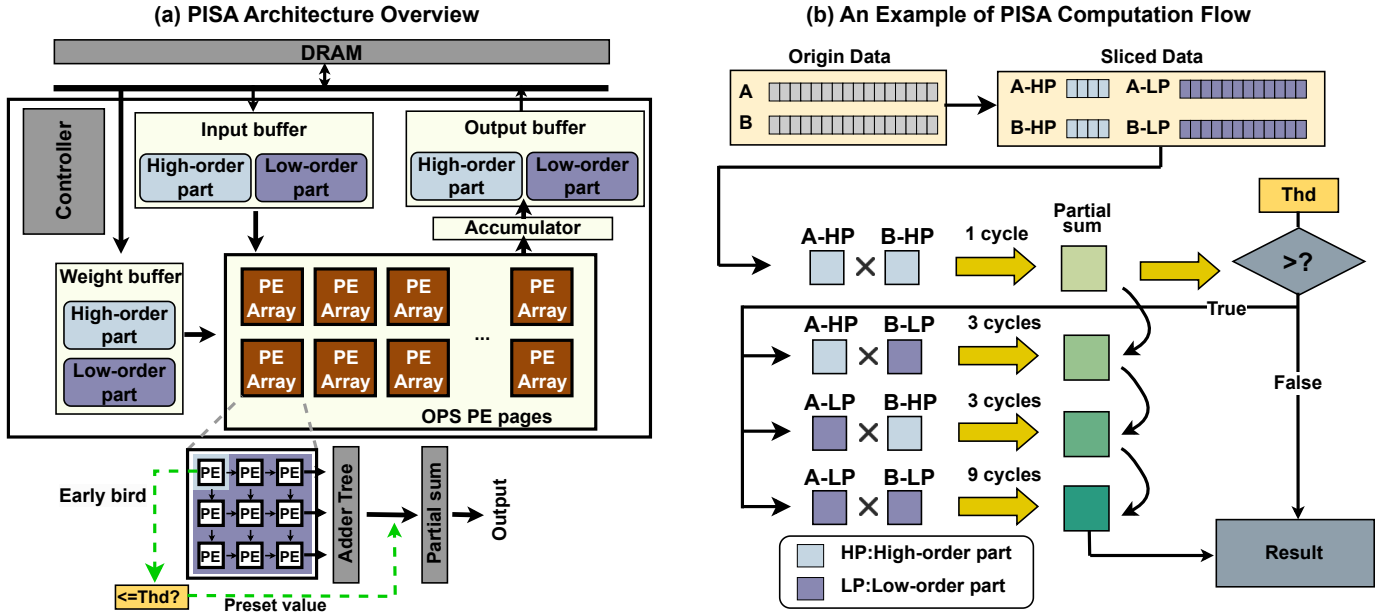


Fig. 5: PISA (a) Architecture Overview and (b) An Example of Computation Flow.

PE page is equipped with a controller, input buffer, weight buffer, output buffer, partial sum buffer, an accumulator, and associated peripheral circuits with control logic.

A key feature of the PISA architecture is its precise categorization of model parameters according to precision-slicing. Parameters are divided into higher-order and lower-order parts based on the slicing strategy and stored in their respective buffers. These parts remain separate and are only intertwined during computation. After computation, they are again separated before output, and not merged until just before the final output is performed.

This design leverages the sparsity and importance of the high-order part for computational acceleration. By keeping high and low-order parts separate, PISA efficiently applies its early bird mechanism, skipping unnecessary computations. This approach leads to significant performance gains without compromising accuracy.

B. PE Design

The PISA computation process shares similarities with conventional systolic arrays but introduces two key differences: 4-bit precision computation and an early bird acceleration mechanism. Figure 5(a) illustrates this design.

PE arrays efficiently process inputs and weights independently, feeding results into an adder tree and summarizing them in a partial sum buffer for final output. We implement a simple logical modification to prioritize high-order part computations, allowing straightforward output from a single PE for comparison with a fixed threshold Thd . When computation results fall below or equal to this threshold, the early bird mechanism activates, bypassing remaining high-order with low-order and low-order with low-order operations. Instead, it inputs a preset value to the partial sum buffer.

To demonstrate the early bird mechanism's efficacy, Figure 5(b) depicts the PISA architecture's computational flow. Consider a scenario with a single 4-bit PE for sliced data: tradi-

tional computation requires 16 cycles to process two numbers (A and B) and sum them. In contrast, PISA computes higher-order parts of A and B in the first cycle, comparing results against the threshold to determine whether to continue. If interrupted, the subsequent 15 computation cycles are skipped. For systolic arrays requiring seven cycles, PISA can potentially save six cycles.

This PE design exploits both the sparsity and the significance of higher-order bits in 16-bit computations. It often requires only a single 4-bit computation to obtain results in most cases, leveraging the fact that higher-order bits contribute more significantly to the final outcome. As a result, this approach substantially reduces computational overhead and energy consumption, thereby enhancing overall efficiency.

C. PISA Execution Flow

Figure 5 illustrates the execution flow of PISA computation, which comprises five steps:

- **Data Slicing:** Input data is partitioned into 4-bit high-order and 12-bit low-order components. This division occurs directly in storage using dedicated buffers, eliminating the need for additional encoding processes.
- **Weight Preparation:** Weights are pre-partitioned into high-order and low-order components and stored in their respective buffers. These pre-processed weights are then fixed in the systolic array prior to computation.
- **Early Bird Computation:** The partitioned inputs and weights enter the PE array in 4-bit format. For each computation set, PISA initially processes the high-order parts and determines whether to activate the early termination mechanism. If activated, the remaining computations in the set are bypassed, and predetermined values are used to populate the partial sum buffers. This approach leverages the sparsity of higher-order data to significantly accelerate computation.

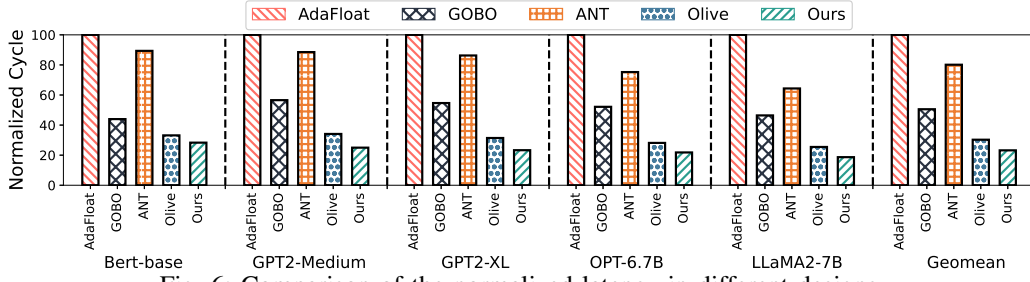


Fig. 6: Comparison of the normalized latency in different designs.

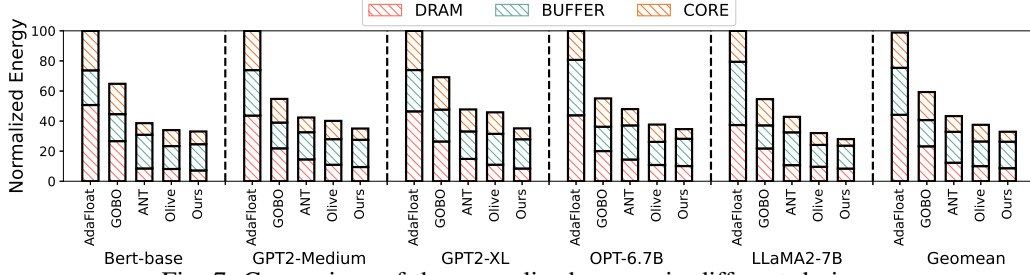


Fig. 7: Comparison of the normalized energy in different designs.

TABLE I: Accuracy and Perplexity of LLMs with PTQ. Lower perplexity (PPL) indicates better performance.

Model	BERT	GPT2-medium	GPT2-XL	OPT-6.7B	LLaMA-7B
Dataset	SST-2 (Acc.) $\uparrow$	Wikitext-103 (PPL) $\downarrow$			
FP32	93.23	22.76	16.94	10.86	5.68
Olive 8/16-bit [11]	92.88	25.42	19.49	18.34	36.47
PISA 8/16-bit	92.77	23.55	18.15	13.11	8.98

- **Partial Sum Accumulation:** Rather than directly aggregating results from each case, PISA accumulates partial sums in a dedicated buffer. This strategy ensures efficient management of intermediate results.
- **Precision-Aware Reduction:** PISA performs a reduction operation on the accumulated partial sums to generate 16-bit output results. These results are then stored in an interleaved format, with the 4-bit high-precision and 12-bit low-precision parts separated before being transferred to the output buffer.

PISA efficiently handles LLMs with outliers by exploiting the inherent sparsity in high-order bits introduced by these outliers. This approach, combined with selective computation skipping, achieves significant performance improvement without incurring additional costs. Moreover, due to the absence of coding and decoding mechanisms, PISA can be seamlessly integrated into existing accelerators, such as systolic arrays.

V. EVALUATION

A. Methodology

Benchmark. To evaluate the performance of the PISA framework, we implement it using the PyTorch [21] framework. We select several widely-accepted transformers: Bert, GPT2-medium, GPT2-XL, OPT-6.5B [22], and LLaMA-7B. Bert is tested on the GLUE dataset suite [23], and the remaining models are validated using the Wikitext-103 dataset [24]. Additionally, we implement the proposed PISA architecture in Verilog and synthesize it using SynPISAs Design Compiler

[25] to estimate chip area and total power consumption under 45nm technology at 500MHz [26]. We also use CACTI [27] to model the energy and area of each on-chip buffer.

Baseline. We compare PISA with the following four baselines: 1) AdaptivFloat [28], or AdaFloat as short, which dynamically clips parameter range at a layer granularity to create faithful encodings, and requires high-precision float to maintain the original model accuracy; 2) GOBO [10], which employs a list of coordinates to indicate the locations of different data representations, using high-precision quantization for larger data representations. 3) ANT [29], which is a hardware-friendly quantization accelerator that combines the power-of-two and INT data type for low bit-width; 4) Olive [11], which is outlier-aware quantization accelerator. We reproduce the method based on the methods provided in these papers and expand the data format from 8 bits to 16 bits to make fair comparisons.

B. Experimental Results

Accuracy/Perplexity. Table I shows the evaluation results of PISA on the SST-2 and Wikitext-103 datasets. The model under the PISA framework maintains very low accuracy loss and excellent perplexity. These results demonstrate that even with most computations skipped through the early bird mechanism, PISA can maintain model accuracy while providing acceleration for LLMs. To summarize, our PISA quantization framework pushes the limit of quantization to a new state-of-the-art, as it achieves nearly original accuracy for commonly used LLMs while significantly reducing computational costs.

Performance. Figure 6 illustrates the normalized total execution cycles across different accelerators for five networks. PISA demonstrates superior performance improvement, achieving substantial speedups with average improvements of $4.31\times$, $2.19\times$, $3.45\times$, and $1.30\times$ over AdaFloat, GOBO, ANT, and Olive, respectively. Notably, PISA achieves a $3.70\times$ speedup over ANT on GPT2-XL. GOBO and ANT perform less effectively on most models, especially struggling with

TABLE II: The core’s configuration and area breakdown of PISA and other work under 45nm process.

Architecture	Core		
	Component	Number	Area(mm ²)
PISA	Comparator (3.5 μm^2)	64	0.387
	8-bit PE(377.5 μm^2)	1024	
Olive [11]	8-bit Decoder(245.5 μm^2)	64	0.412
	16-bit Decoder(320.4 μm^2)	32	
	8-bit PE(377.5 μm^2)	1024	
ANT [29]	Decoder(24.2 μm^2)	64	0.388
	8-bit PE(377.5 μm^2)	1024	
GOBO [10]	8-bit PE(377.5 μm^2)	1024	0.450
	16-bit SPU (996.0 μm^2)	64	
AdaFloat [28]	8-bit PE(377.5 μm^2)	1024	0.387

attention-based models due to their complex encoding and decoding mechanisms. PISA stands out by fully exploiting inherent sparsity through precision-slicing and achieving significant computational speedup via the early-bird mechanism, without additional encoding and decoding overhead.

Energy. Figure 7 shows the normalized energy consumption across five networks, broken down into DRAM, global buffer (BUFFER) and processing core (CORE). PISA exhibits the lowest energy consumption for all transformers, consuming on average 66.7%, 44.5%, 24.0%, and 14.3% less energy compared to AdaFloat, GOBO, ANT, and Olive, respectively. This energy advantage stems from PISA avoiding complex encoder/decoder mechanisms and leveraging the higher-order sparsity and importance in LLMs to skip computations.

Area. Table II compares the area and number of PEs for PISA and other accelerators, all scaled to a 45-nm process, including decoder areas where applicable. PISA achieves the lowest area overhead by using compact 4-bit PEs without decoding mechanisms and implementing the early bird mechanism with a single comparator. In contrast, other accelerators like GOBO incur substantial area overhead due to the inclusion of decoders. This design allows PISA to accommodate more PEs within the same area budget, improving throughput.

C. Sensitivity Analysis

This section examines the impact of the early bird mechanism on PISA, focusing on how different thresholds and preset

TABLE III: Perplexity results with Different Threshold for GPT2-XL. Preset values are layer-wise optimized.

Threshold	Origin	0	3	7	15	31	Layer-wise Optimization
Percentage of early birds(%)	-	87.6	87.9	88.6	90.4	92.8	91.3
PPL	16.94	18.04	20.12	25.66	89.27	766.82	18.15

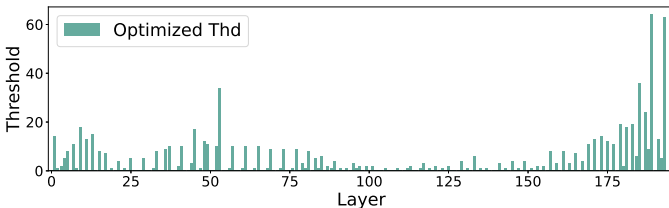


Fig. 8: Optimized layer-wise Threshold of GPT2-XL.

TABLE IV: Perplexity results with Different Strategies of Preset Value for GPT2-XL. Threshold are layer-wise optimized.

Strategy of preset value	Origin	All set to 0	Mean Value of each layer	Optimization after search
PPL	16.94	135.15	42.49	18.15

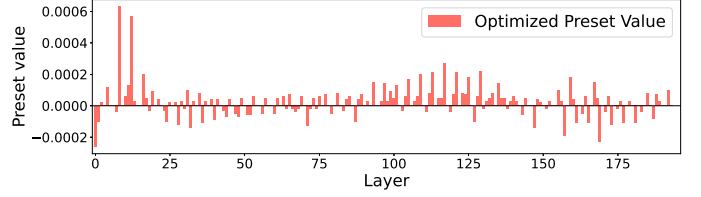


Fig. 9: Optimized layer-wise Preset Values of GPT2-XL.

values affect performance and accuracy.

Impact of Threshold. The early bird mechanism in PISA skips remaining computations if the product of two 4-bit higher-order parts is below a threshold, outputting a preset value instead. Choosing an appropriate threshold is crucial to balance accuracy and performance. Table III shows the computation saving and perplexity results for fixed thresholds (0 and $2^n - 1$) on the GPT2-XL model. To optimize, we performed a layer-wise binary search for thresholds that minimize MSE on the validation set. Figure 8 displays the optimal thresholds for each layer. This approach allows PISA to exploit computation redundancy in insensitive layers, achieving higher performance than fixed thresholds with negligible accuracy loss.

Impact of preset value. The early bird mechanism uses a preset value to preserve information when skipping computations. Choosing a suitable preset value is crucial for maintaining model accuracy. Table IV shows the evaluation of several strategies. Setting all preset values to 0 significantly degrades inference performance, similar to using a fixed threshold. We optimized preset values layer by layer, starting from the mean value of each layer’s parameters and searching a small area around it. This approach successfully maintained low perplexity, as shown in Table IV. Figure 9 shows the distribution of optimized preset values.

Our analysis reveals that layers with both threshold and preset values set to 0 are predominantly linear projections within the attention block. Consequently, for a simplified configuration, we can uniformly set the thresholds and preset values of all linear projection layers in the attention block to 0 while still maintaining strong performance.

VI. CONCLUSION

This paper introduces PISA, an precision-slice framework designed to enhance the efficiency of LLM inference. The key insight of PISA is to exploit the inherent sparsity and contribution within the computation of higher-order parts through the bit-slicing scheme. By integrating an early-bird mechanism, PISA selectively bypasses unnecessary computations for better performance. Evaluations show that PISA delivers significant performance gains with only a negligible accuracy loss.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] T. B. Brown, “Language models are few-shot learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [3] F. Liu, W. Zhao, Z. Wang, Y. Chen, Z. He, N. Jing, X. Liang, and L. Jiang, “Ebsp: evolving bit sparsity patterns for hardware-friendly inference of quantized deep neural networks,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 259–264.
- [4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [5] S. Sharify, Z. Xu, X. Wang *et al.*, “Combining multiple post-training techniques to achieve most efficient quantized llms,” *arXiv preprint arXiv:2405.07135*, 2024.
- [6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [7] F. Liu *et al.*, “Improving neural network efficiency via post-training quantization with adaptive floating-point,” in *ICCV*, 2021.
- [8] M. van Baalen, A. Kuzmin, M. Nagel, P. Couperus, C. Bastoul, E. Mahurin, T. Blankevoort, and P. Whatmough, “Gptvq: The blessing of dimensionality for llm quantization,” *arXiv preprint arXiv:2402.15319*, 2024.
- [9] E. Park, D. Kim, and S. Yoo, “Energy-efficient neural network accelerator based on outlier-aware low-precision computation,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 688–698.
- [10] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
- [11] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [12] X. Ma, S. Lin, S. Ye, Z. He, L. Zhang, G. Yuan, S. H. Tan, Z. Li, D. Fan, X. Qian *et al.*, “Non-structured dnn weight pruning—is it beneficial in any platform?” *IEEE transactions on neural networks and learning systems*, vol. 33, no. 9, pp. 4930–4944, 2021.
- [13] F. Liu, N. Yang, H. Li, Z. Wang, Z. Song, S. Pei, and L. Jiang, “Spark: Scalable and precision-aware acceleration of neural networks via efficient encoding,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 1029–1042.
- [14] F. Liu, N. Yang, Z. Song, Z. Wang, H. Li, S. Huang, Z. Song, S. Pei, and L. Jiang, “Inspire: Accelerating deep neural networks via hardware-friendly index-pair encoding,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [15] X. Wei, Y. Zhang, X. Zhang, R. Gong, S. Zhang, Q. Zhang, F. Yu, and X. Liu, “Outlier suppression: Pushing the limit of low-bit transformer language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 402–17 414, 2022.
- [16] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 168–27 183, 2022.
- [17] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “Qserve: W4a8kv4 quantization and system co-design for efficient llm serving,” *arXiv preprint arXiv:2405.04532*, 2024.
- [18] H. Xia, Z. Zheng, X. Wu, S. Chen, Z. Yao, S. Youn, A. Bakhtiari, M. Wyatt, D. Zhuang, Z. Zhou *et al.*, “Fp6-llm: Efficiently serving large language models through fp6-centric algorithm-system co-design,” *arXiv preprint arXiv:2401.14112*, 2024.
- [19] N. Yang, F. Liu, Z. Wang, J. Zhao, and L. Jiang, “Searchq: Search-based fine-grained quantization for data-free model compression,” *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, 2024.
- [20] G.-M. Tang, K. Takata, M. Tanaka, A. Fujimaki, K. Takagi, and N. Takagi, “4-bit bit-slice arithmetic logic unit for 32-bit rsfq microprocessors,” *IEEE Transactions on Applied Superconductivity*, vol. 26, no. 1, pp. 1–6, 2015.
- [21] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [22] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [23] A. Wang, “Glue: A multi-task benchmark and analysis platform for natural language understanding,” *arXiv preprint arXiv:1804.07461*, 2018.
- [24] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
- [25] S. D. Compiler, [Online]. Available: <https://www.synopsys.com/support/training/rtlsynthesis/design-compiler-rtl-synthesis.html>, 2019.
- [26] D. Fujiki, S. Mahlke, and R. Das, “In-memory data parallel processor,” *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 1–14, 2018.
- [27] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “Cacti 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [28] T. Tamba, E.-Y. Yang, Z. Wan, Y. Deng, V. J. Reddi, A. Rush, D. Brooks, and G.-Y. Wei, “Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [29] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, “Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.